Final Project Report

Banking System in Go

Team: Shynarbek Nuraziz , Asset Kamza

GitHub: https://github.com/Qpzm1212/FInal-SDP-Banking-SyStem.git

1. Introduction

The project implements topic 11 of the assignment, Banking System, as a working web application written in Go. The topic suggests Strategy for interest calculation, Builder for loan agreements and Observer for fraud alerts. We added Factory Method for opening accounts, Singleton for the storage and Facade as the single entry point, so six design patterns work together in one project.

Goal. Show that each pattern solves a real problem of the banking domain and that all six are operational in one cohesive application.

Tasks. Open accounts of three types (checking, savings, credit); transfer money with balance checks; detect suspicious transfers; issue loan agreements with a payment schedule summary; calculate interest by different schemes; store data persistently; give the user a modern web interface.

Technologies. Go, Echo v4 (HTTP), a JSON file as storage, web application and the standard Go testing package.

2. System Overview

2.1 Architecture

The application has five layers. The browser contains no business logic: it only calls the REST API.

```
web/index.html            single-page UI (Home, Accounts, Transfer, Loan, Calculator)
cmd/api/main.go           HTTP handlers, error-to-status mapping
internal/service/         Facade
internal/patterns/        Factory Method, Builder, Strategy, Observer
internal/db/              Singleton storage (JSON file)
internal/models/          data models
```

2.2 REST API

Endpoint	Purpose	Pattern
GET /api/accounts	list accounts with balances	Facade, Singleton
POST /api/account	open an account (type, id)	Factory Method
GET /api/transactions	transaction history, newest first	Facade, Singleton
POST /api/transaction	transfer (account_id, amount)	Observer

Endpoint	Purpose	Pattern
POST /api/loan	create a loan agreement (borrower, amount)	Builder
GET /api/interest	calculate yearly interest (type, balance)	Strategy

Errors are returned as JSON with a suitable HTTP status: 400 invalid data, 404 account not found, 409 account already exists, 422 insufficient funds.

2.3 Behaviour of the System

On first start the server creates two demo accounts (ACC-1 checking, ACC-2 savings) with a short history. Checking and savings accounts cannot go below zero, while credit accounts can (the negative balance is the debt). A transfer above 10000 is accepted but flagged as suspicious, and the web page shows a warning and a notification badge.

3. Design Patterns

3.1 Summary Table

#	Pattern	Where in the code	What it does	Visible in the application
1	Singleton	internal/db/singleton.go: Database, GetDBInstance (sync.Once, RWMutex)	One shared thread-safe storage; atomic debit with balance rules; JSON persistence	every page reads and writes through it
2	Factory Method	internal/patterns/factory.go: AccountCreator, CheckingCreator, SavingsCreator, CreditCreator	Each creator builds its own account type; the type code selects the creator	Accounts page
3	Builder	internal/patterns/builder.go: LoanBuilder, StandardLoanBuilder, LoanDirector	Assembles a loan agreement step by step, validates it and calculates the monthly payment	Loan page (3-step form)
4	Strategy	internal/patterns/strategy.go: InterestStrategy, SavingsStrategy, LoanStrategy, InterestContext	Interchangeable interest formulas (5% savings, 15% credit) chosen by name	Calculator page
5	Observer	internal/patterns/observer.go: TransactionMonitor, FraudDetector, AuditLogger,	Sends every transaction event to all subscribers: fraud check, audit log,	Transfer page, notification badge,

#	Pattern	Where in the code	What it does	Visible in the application
		SecurityTeamAlert	security alert	Security Monitor
6	Facade	internal/service/facade.go: BankFacade	One simple API over all subsystems; validation and error mapping	all endpoints

3.2 Code Excerpts

Singleton: one instance, created once.

```
func GetDBInstance() *Database {
    once.Do(func() {
        path := os.Getenv("BANK_DB_PATH")
        if path == "" {
            path = defaultPath
        }
        instance = newDatabase(path)
    })
    return instance
}
```

Factory Method: a new account type is one creator and one line in the registry.

```
var creators = map[string]AccountCreator{
    "checking": CheckingCreator{},
    "savings":  SavingsCreator{},
    "credit":   CreditCreator{},
}
```

Observer: the detector decides, other observers react to the mark.

```
func (f *FraudDetector) Update(e *TransactionEvent) {
    if e.Amount > f.Threshold {
        e.Suspicious = true
    }
}
```

Facade: a transfer touches Singleton and Observer, the caller sees one method.

```
event := f.Monitor.ProcessTransaction(amount, accID)
rec := models.TransactionRecord{AccountID: accID, Name: "Transfer",
    Amount: amount, Suspicious: event.Suspicious, Date: event.Time}
balance, err := dbInst.Debit(rec)
```

3.3 How the Patterns Work Together

Scenario	Flow
Open an account	Facade.OpenAccount, then Factory Method creates the account, then Singleton saves it and rejects a duplicate number
Transfer	Facade.Transfer, then Singleton checks the account and funds, then

Scenario	Flow
	Observer (FraudDetector marks, AuditLogger logs, SecurityTeamAlert alerts), then Singleton debits and stores the record under one lock
Loan	Facade.ApplyForLoan, then LoanDirector drives the Builder, then the validated agreement is saved through Singleton
Interest	Facade.CalculateYield, then Strategy is selected by name and executed in the context

Direct interaction between patterns: Builder and Strategy share the constant StandardLoanRatePercent, so the loan rate cannot diverge; the Facade configures the Observer and registers the observers in the correct order (the detector first).

4. Division of Work

The team has two students, so each one implemented three patterns on his own (the requirement is at least two).

Student	Patterns	Files	Other work
Shynarbek Nuraziz	Singleton, Factory Method, Facade	db/singleton.go, patterns/factory.go, service/facade.go	models, main.go , db and service tests
Asset Kamza	Builder, Strategy, Observer	patterns/builder.go, patterns/strategy.go, patterns/observer.go	web/index.html , pattern tests

5. Clean Code Principles Used

Principle	How it is applied	Where
Single Responsibility	Handlers only translate HTTP, the facade orchestrates, patterns hold domain logic, the db package only stores data	main.go, facade.go, patterns, singleton.go
Open/Closed	A new account type, interest scheme or observer is added without changing existing code	creators and strategies maps, NewBankFacade
Program to interfaces	Account, AccountCreator, InterestStrategy, Observer, LoanBuilder	internal/patterns
DRY	One loan rate constant for Builder and Strategy; shared helpers fail, parseNumber, api, post	strategy.go, main.go, index.html
No magic numbers	Named constants FraudThreshold, SavingsRatePercent, StandardLoanRatePercent,	observer.go, strategy.go, singleton.go

Principle	How it is applied	Where
	CreditType	
Meaningful names	FraudDetector, AuditLogger, ConstructStandardLoan, ErrInsufficientFunds	whole code base
Fail fast, explicit errors	Input is validated at the boundary; sentinel errors are wrapped with %w; one function maps errors to HTTP codes	patterns, facade.go, main.go
Encapsulation and thread safety	Database data is private and reachable only through methods under RWMutex; Notify iterates over a copy of the observer list	singleton.go, observer.go
Consistency and defensive copying	Build returns a copy; a failed save rolls back the in-memory change; the file is written to a temporary file and renamed	builder.go, singleton.go
Testability	Storage path set by BANK_DB_PATH, constructor newDatabase(path), short single-purpose functions	singleton.go, tests

6. Testing

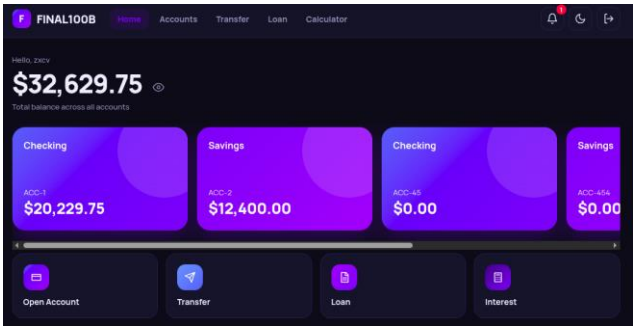
File	Tests	What they check
internal/patterns/patterns_test.go	TestStrategySelection, TestBuilderValidationAndPayments, TestFactoryMethod, TestObserverNotifiesAll	interest 5% and 15%, unknown type error, loan validation and payment, account creation, events reach all observers and removal works
internal/db/db_test.go	TestSeedAndPersistence, TestDuplicateAccount, TestDebitRules, TestConcurrentDebits	demo data and reload from file, duplicate number, insufficient funds, credit overdraft, 50 concurrent debits give the exact balance
internal/service/facade_test.go	TestTransferFlow, TestAccountsAndLoansAndInterest	full scenarios through the facade, including the suspicious flag and error types

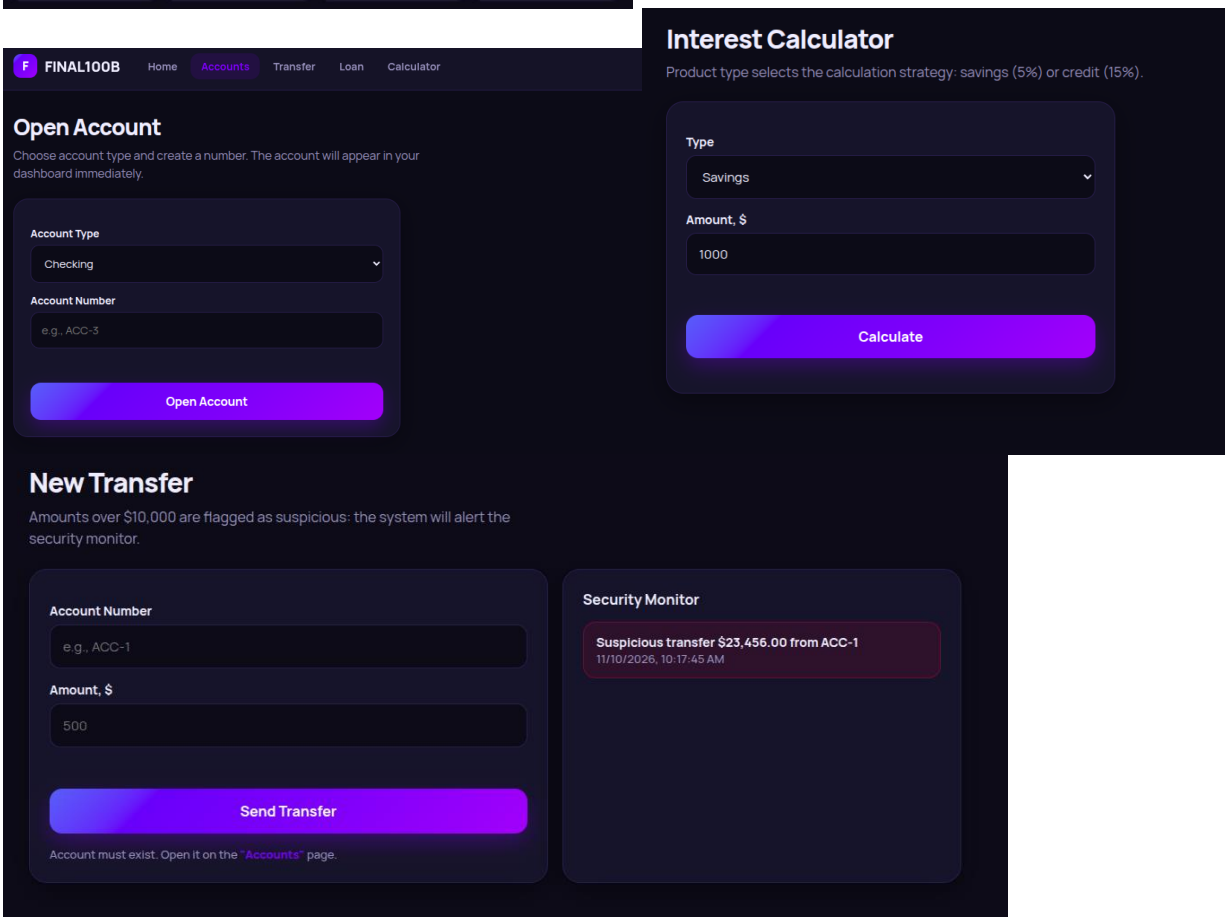
Result of test

```
><3 go setup calls>
?       ESHKEREE/cmd/api    [no test files]
=== RUN   TestSeedAndPersistence
--- PASS: TestSeedAndPersistence (0.00s)
=== RUN   TestDuplicateAccount
--- PASS: TestDuplicateAccount (0.00s)
=== RUN   TestDebitRulesAndCreditLimit
--- PASS: TestDebitRulesAndCreditLimit (0.00s)
=== RUN   TestConcurrentDebits
--- PASS: TestConcurrentDebits (0.03s)
PASS
ok      ESHKEREE/internal/db    (cached)
?       ESHKEREE/internal/models    [no test files]
=== RUN   TestStrategySelection
--- PASS: TestStrategySelection (0.00s)
=== RUN   TestBuilderValidationAndPayments
--- PASS: TestBuilderValidationAndPayments (0.00s)
=== RUN   TestFactoryMethod
--- PASS: TestFactoryMethod (0.00s)
=== RUN   TestObserverNotifiesAndDetectsVelocity
--- PASS: TestObserverNotifiesAndDetectsVelocity (0.00s)
PASS
ok      ESHKEREE/internal/patterns  (cached)
=== RUN   TestTransferFlow
[AUDIT] 19:34:46 | account ACC-1 | $500.00 | OK
[AUDIT] 19:34:46 | account ACC-1 | $15000.00 | FLAGGED
[ALERT] Suspicious transaction $15000.00 from account ACC-1 (exceeds $10000 AML limit)! Security team
--- PASS: TestTransferFlow (0.00s)
=== RUN   TestAccountsAndLoansAndInterest
[AUDIT] 19:34:46 | account CR-9 | $300.00 | OK
--- PASS: TestAccountsAndLoansAndInterest (0.00s)
PASS
ok      ESHKEREE/internal/service   (cached)

Process finished with the exit code 0
```

7. Screenshots





8. Conclusion

We built a working banking web application in which six design patterns (Singleton, Factory Method, Builder, Strategy, Observer and Facade) cooperate through one facade. Each pattern has a concrete job in the domain, the business logic lives only on the Go server, the system validates input and returns clear errors, and ten unit tests cover storage, patterns and the facade. The work was shared between two students, three patterns each.

Limitations: the sign-in page on the site is a simulation (the user is kept in the browser and the data is shared by all visitors), the storage is a JSON file, and loans are not yet connected to account balances.

9. Further Work

- real authentication with per-user accounts;
- Decorator for transfer fees and extra checks, as a seventh pattern;
- replace the JSON file with PostgreSQL behind the same db interface;
- incoming transfers and deposits, and loan payments from an account;
- more observers (e-mail and SMS) and fraud rules by number of operations per period;
- Docker image and automated end-to-end tests.

10. References

1. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
2. Freeman E., Robson E. Head First Design Patterns. O'Reilly Media.
3. Martin R. Clean Code. Prentice Hall, 2008.
4. Refactoring.Guru. Design Patterns. https://refactoring.guru/design-patterns
5. Go documentation. https://go.dev/doc
6. Echo documentation. https://echo.labstack.com